

INSTITUTO TECNOLÓGICO Y DE ESTUDIOS SUPERIORES DE OCCIDENTE

DEPARTAMENTO DE ELÉCTRONICA, SISTEMAS E INFORMÁTICA

PROGRAMACIÓN ORIENTADA A OBJETOS



ITESO, Universidad
Jesuita de Guadalajara

TAREA #5 — CLASE DATE

PRESENTA

Viviana Toledo De la Fuente

Hugo Iván Piza Dávila

Fecha: 4/10/2026

Clase Date

1. Constructores

La clase Date consta de 5 atributos: day, month, year, monthName y format. Cada uno tiene un valor de inicialización por default. Aunque algunos pueden resultar redundantes (como monthName), decidí dejarlos inicializados.

La clase contiene 3 constructores, uno sin argumentos, otro con 3 argumentos y otro con 4 argumentos, incluyendo el formato de impresión de la fecha. El constructor sin argumentos crea la fecha 01/01/2026, mientras que los demás utilizan los valores ingresados por el usuario. Cada constructor llama al Zero, que utiliza los *Setters*.

```
You, 59 minutes ago | author (You)
package model;

You, 59 minutes ago | author (You)
public class Date {
    // Default Values
    int day = 1, month = 1, year = 1900;
    String monthName = "Enero";
    int format = 0;

    // =====
    // Constructors
    // =====

    // No values
    public Date() {
        this(day: 1, month: 1, year: 2026, format: 0);
    }

    // Receives 3 values except format
    public Date(int day, int month, int year) {
        this(day, month, year, format: 0);
    }

    // Receives all values
    public Date(int day, int month, int year, int format) {
        setDay(day);
        setMonth(month);
        setYear(year);
        setFormat(format);
    }
}
```

2. Setters

El primer setter es para day, el cual evalúa si el día entra en el rango válido de cada mes. En caso de que el día sea mayor o igual a 1, y menor o igual al día máximo, el día es seteado. Caso contrario, se queda con el valor default (1).

```

// =====
// Setters
// =====

public void setDay(int day) {
    int maxDay = switch(this.month) {
        case 2 -> 28;
        case 4, 6, 9, 11 -> 30;
        default -> 31; // Since the default is January & January has 31 days
    };

    if (1 <= day && day <= maxDay) {
        this.day = day;
    }
}

```

El setter month evalúa si el mes está dentro del rango válido [1, 12], en caso de que sí, el mes se actualiza. De lo contrario, el mes queda con el valor default (1). A su vez, esta función cambia monthName utilizando un switchcase.

```

public void setMonth(int month) {
    if (1 <= month && month <= 12) {
        this.month = month;

        // Update monthName
        this.monthName = switch(this.month) {
            case 2 -> "Febrero";
            case 3 -> "Marzo";
            case 4 -> "Abril";
            case 5 -> "Mayo";
            case 6 -> "Junio";
            case 7 -> "Julio";
            case 8 -> "Agosto";
            case 9 -> "Septiembre";
            case 10 -> "Octubre";
            case 11 -> "Noviembre";
            case 12 -> "Diciembre";
            default -> "Enero";
        };
    }
}

```

El setter de format y year funcionan de manera similar, evalúan si el dato ingresado es válido para cambiarlo, y si no, no lo cambian.

```

public void setYear(int year) {
    if (1900 <= year && year <= 3000) {
        this.year = year;
    }
}

public void setFormat(int format) {
    if (0 <= format && format <= 2) {
        this.format = format;
    }
}

```

Los getters simplemente devuelven los valores:

```
// =====  
// Getters  
// =====  
  
public int getDay() {  
    return this.day;  
}  
  
public int getMonth() {  
    return this.month;  
}  
  
public int getYear() {  
    return this.year;  
}  
  
public String getMonthName() {  
    return this.monthName;  
}  
  
public int getFormat() {  
    return this.format;  
}
```

El método toString evalúa el formato elegido (donde 0 es el default) e imprime los resultados de acuerdo al formato.

```
// =====  
// Methods  
// =====  
  
// String  
@Override  
public String toString() {  
    return switch(format) {  
        case 1 -> String.format(format: "%d-%s-%d", day, monthName.substring(beginIndex: 0, endIndex: 3), year);  
        case 2 -> String.format(format: "%d de %s de %s", day, monthName.toLowerCase(), year);  
        default -> String.format(format: "%02d/%02d/%02d", day, month, year % 100);  
    };  
}
```

El método equals evalúa si el objeto ingresado es parte de la clase, después lo parsea a la clase Date, y evalúa si ambos lados son iguales. El método clone simplemente hace una copia del objeto.

```
// Equals  
@Override  
public boolean equals(Object obj) {  
    // En caso de que no pertenezca a la clase  
    if (!(obj instanceof Date)) return false;  
  
    // Parsear a Date  
    Date d = (Date) obj;  
  
    // Checar si es igual  
    return  
        d.getDay() == this.day &&  
        d.getMonth() == this.month &&  
        d.getYear() == this.year;  
}  
  
// Clone  
public Date clone() {  
    return new Date(this.day, this.month, this.year, this.format);  
}
```

El método `next()` le suma un día al objeto `Date` actual. Primero, vuelve a calcular el `maxDay` permisible. Si el día del objeto es menor al día máximo, le suma uno. En caso de que sea igual [que sea mayor no está permitido por el método `setDay()`], `day = 1`.

Después, evalúa si el mes es menor a 12 [que esté fuera de rango no está permitido por el método `setMonth()`]. Si es menor, solo le suma 1 al mes. Si es mayor, el mes se vuelve 1 y le suma 1 al año.

```
// Get Next Day
public void next() {
    // Get maxDay for each month
    int maxDay = switch(this.month) {
        case 2 -> 28;
        case 4, 6, 9, 11 -> 30;
        default -> 31; // Since the default is January & January has 31 days
    };

    // If current day is smaller than maxDay
    if (this.day < maxDay) {
        this.day++;
    }
    else {
        // Current day IS maxDay
        this.day = 1;

        // Month is less than 12
        if (this.month < 12) {
            this.setMonth(this.month+1);
        }
        // Month IS 12
        else {
            this.setMonth(month: 1);
            this.year++;
        }
    }
}
```

Pruebas de Ejecución

```
package tests;

import model.Date;

public class TestDate {

    public static void main(String args[]) {

        // Initiliazate dates

        Date d1 = new Date();

        Date d2 = new Date(6, 9, 2004, 1);

        Date d3 = new Date(30, 4, 2005, 0);

        Date d4 = new Date(34, 22, 500, 14);
```

```
Date d5 = new Date(28, 2, 8784, 0);
Date d6 = new Date(-8, -12, 1968, 2);

// Print original dates
System.out.println("=====");
System.out.println("Original Dates:");
System.out.println("=====");
System.out.println(d1.toString());
System.out.println(d2.toString());
System.out.println(d3.toString());
System.out.println(d4.toString());
System.out.println(d5.toString());
System.out.println(d6.toString());

// Setters
System.out.println("\n=====");
System.out.println("Try setters:");
System.out.println("=====");
d1.setDay(31);
d1.setMonth(12);
d1.setYear(2016);
d1.setFormat(2);
System.out.printf("New values of d1: %s\n", d1.toString());

// Getters
System.out.println("\n=====");
System.out.println("Try getters:");
```

```

        System.out.println("=====");

        System.out.printf("Values of d2. Day: %d, Month: %d, Year: %d.
MonthName: %s. Format: %d\n",
                        d2.getDay(), d2.getMonth(), d2.getYear(),
d2.getMonthName(), d2.getFormat()

        );

        // Clone

        System.out.println("\n=====");

        System.out.println("Clone:");

        System.out.println("=====");

        Date d7 = d4.clone();

        System.out.printf("D4: %s. D7: %s\n", d4.toString(),
d7.toString());

        System.out.printf("D4 == D7: %b\n", d4 == d7);

        // Equals

        System.out.println("\n=====");

        System.out.println("Equals:");

        System.out.println("=====");

        System.out.printf("D4 EQUALS D7: %b\n", d4.equals(d7));

        System.out.println("\n=====");

        System.out.println("Next Method:");

        System.out.println("=====");

        System.out.printf("Current day: %s\n", d3.toString());

        d3.next();

        System.out.printf("Next day: %s\n", d3.toString());

```

```
System.out.println();  
System.out.printf("Current day: %s\n", d1.toString());  
d1.next();  
System.out.printf("Next day: %s\n", d1.toString());  
}  
}
```

```
=====
Original Dates:
=====
01/01/26
6-Sep-2004
30/04/05
01/01/00
28/02/00
1 de enero de 1968

=====
Try setters:
=====
New values of d1: 31 de diciembre de 2016

=====
Try getters:
=====
Values of d2. Day: 6, Month: 9, Year: 2004. MonthName: Septiembre. Format: 1

=====
Clone:
=====
D4: 01/01/00. D7: 01/01/00
D4 == D7: false

=====
Equals:
=====
D4 EQUALS D7: true

=====
Next Method:
=====
Current day: 30/04/05
Next day: 01/05/05

Current day: 31 de diciembre de 2016
Next day: 1 de enero de 2017
```

Conclusiones

Disfruté bastante de la creación de la clase Date, creo que fue una buena práctica para crear métodos en Java, así como familiarizarme con las particularidades de los objetos y clases. Aprendí a incorporar switch-cases en métodos para ser más rigurosa con las validaciones, así como abreviar el código dentro de los métodos para evitar la verbosidad. No tuve grandes complicaciones, aunque el ejercicio fue retador pues necesité incorporar validaciones para múltiples campos.